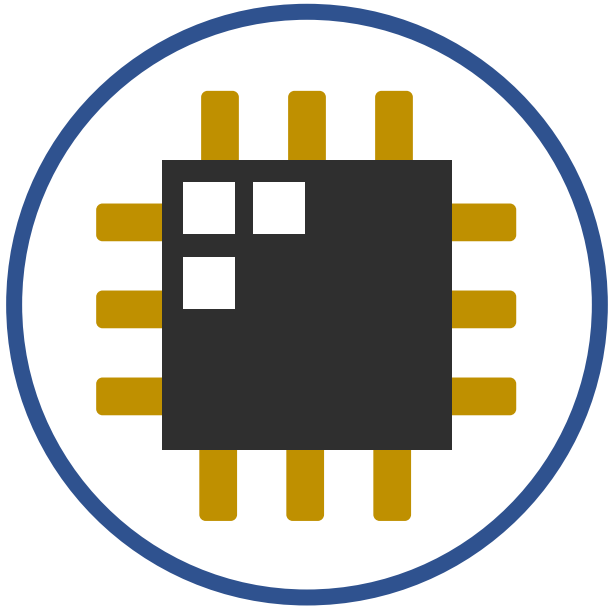
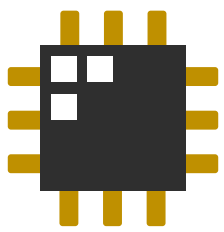




EEE 113:

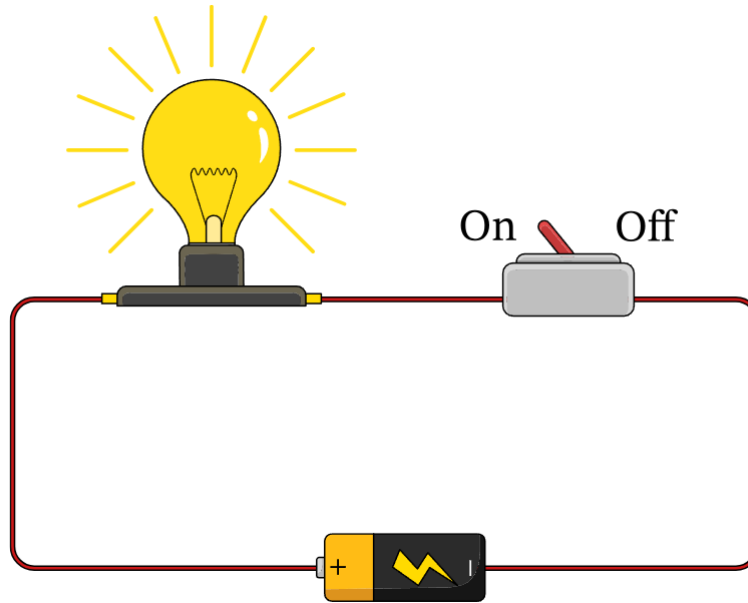
Introduction to Electrical and Electronics Engineering Systems

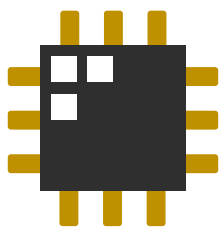
Week 1 – Logic gates, Boolean algebra,
and logical equivalence



Introduction to logic

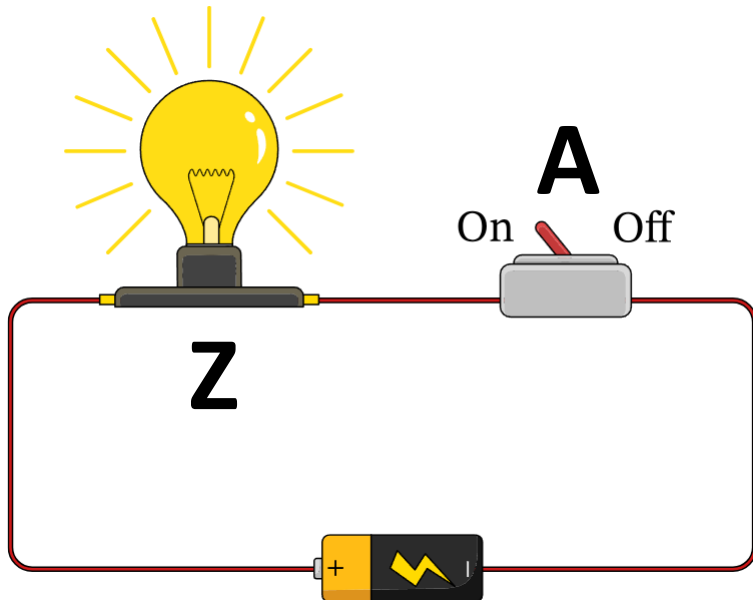
- Let's utilize binary bits and do something cool with them





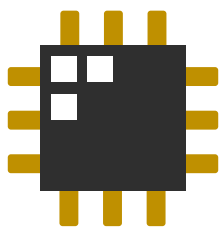
Introduction to logic

- **Switches:** Basic element of a physical circuit
- If A is 1, switch is closed, Z (light bulb) turns on
- If A is 0, switch is open, Z (light bulb) turns off



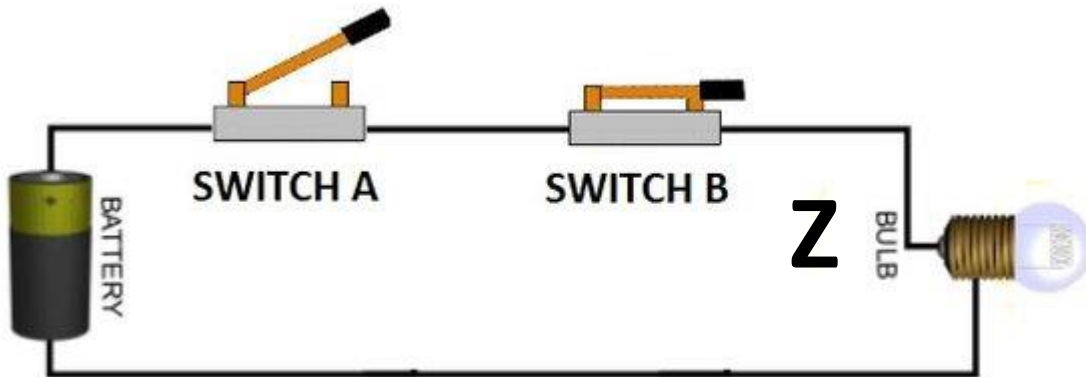
$$Z = A$$

Whatever the state of A is,
Z copies it



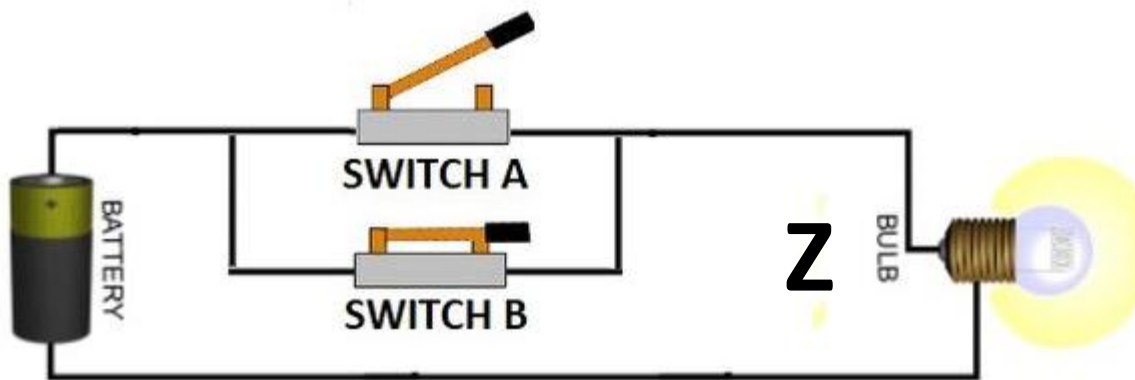
Introduction to logic

- We can make more complex ones!



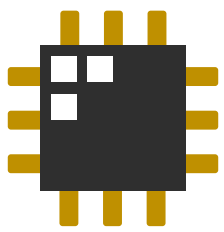
$$Z = A \text{ AND } B$$

A and B needs to be 1 so that Z turns on (1)



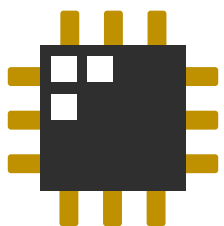
$$Z = A \text{ OR } B$$

A or B needs to be 1 so that Z turns on (1)

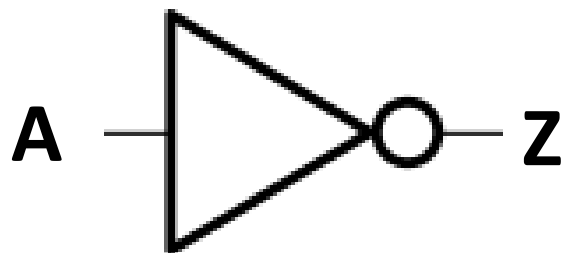


Introduction to logic

- Early computer designers built ad hoc circuits from switches
 - They began to notice common ‘patterns’/logic in their work: ANDs, ORs
- Claude Shannon’s 1938 thesis made link between switches and logic.
 - Shoutout to the Comms section of EEE 113!
- George Boole came up with a linguistic algebra to encode logic into a language that can be manipulated and solved mathematically
 - We call it *Boolean logic* in his honor.



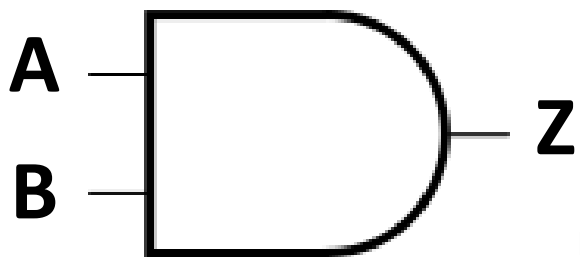
Introduction to logic



NOT gate

$$Z = A'$$

We usually say "A prime", or "not A"

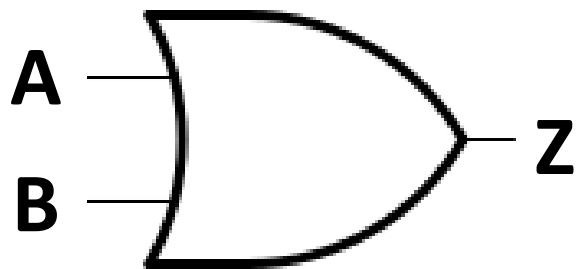


AND gate

$$Z = AB$$

We usually say "A and B"

In a way it is like binary multiplication, right?



OR gate

$$Z = A + B$$

We usually say "A or B"

Don't be fooled, this is not addition!

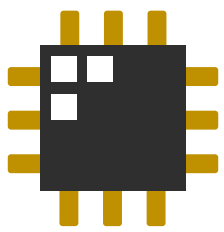
A	Z
0	1
1	0

These are what we call **truth tables**

What is the output given the input/s combination?

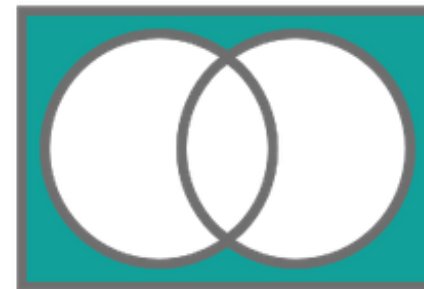
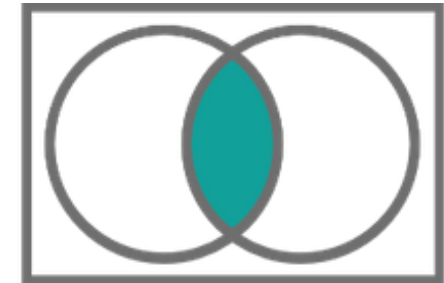
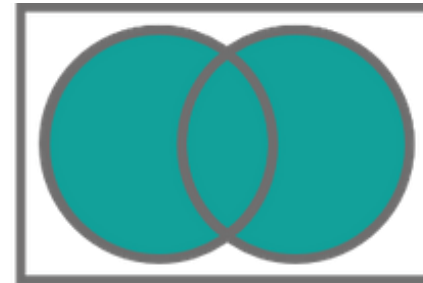
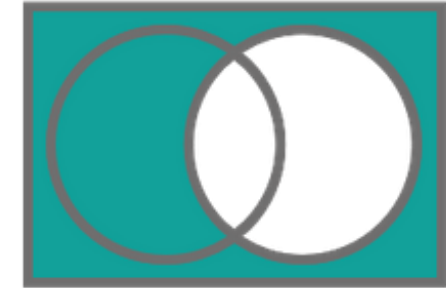
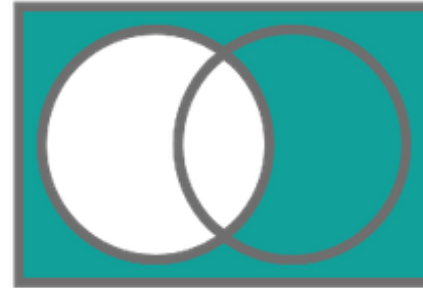
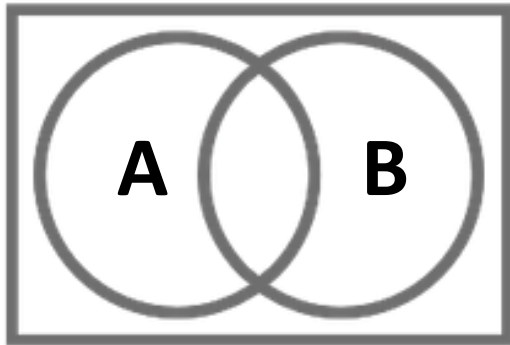
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

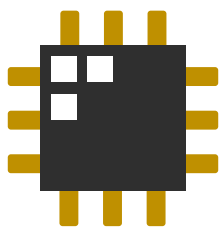
A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1



Introduction to logic

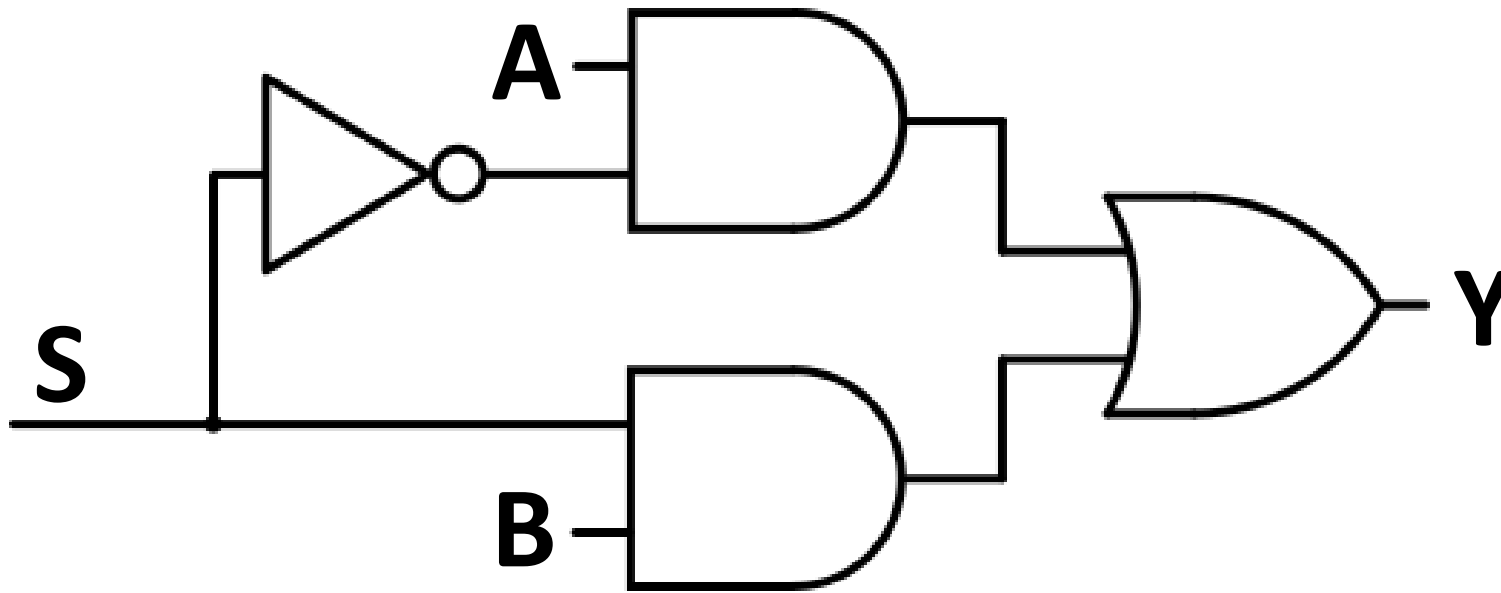
- Boolean logic is actually just set theory!
 - Can you identify the logic being implied in the following diagrams? (shaded means true/1)



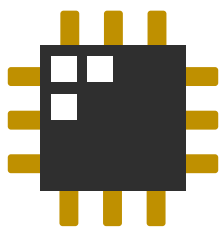


More complex logic

- We can make combinations of logic gates to make more complex logic!

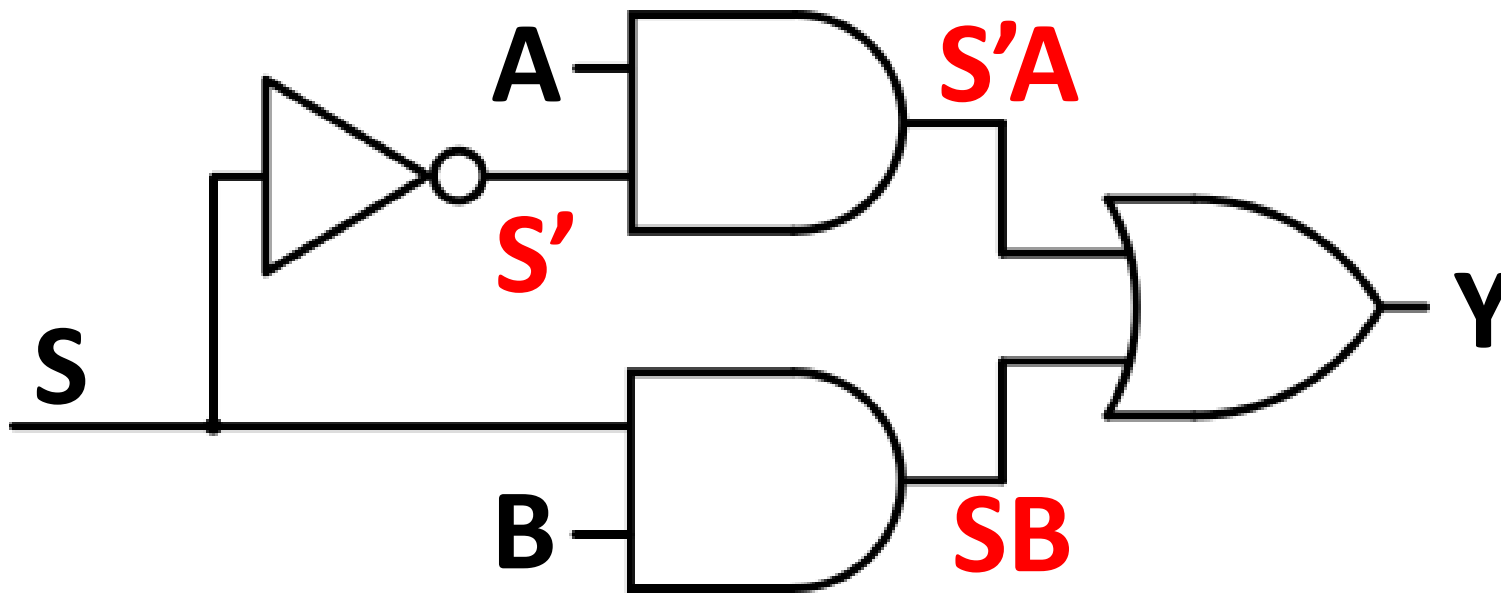


S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	



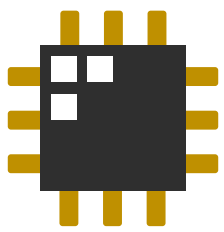
More complex logic

- We can make combinations of logic gates to make more complex logic!



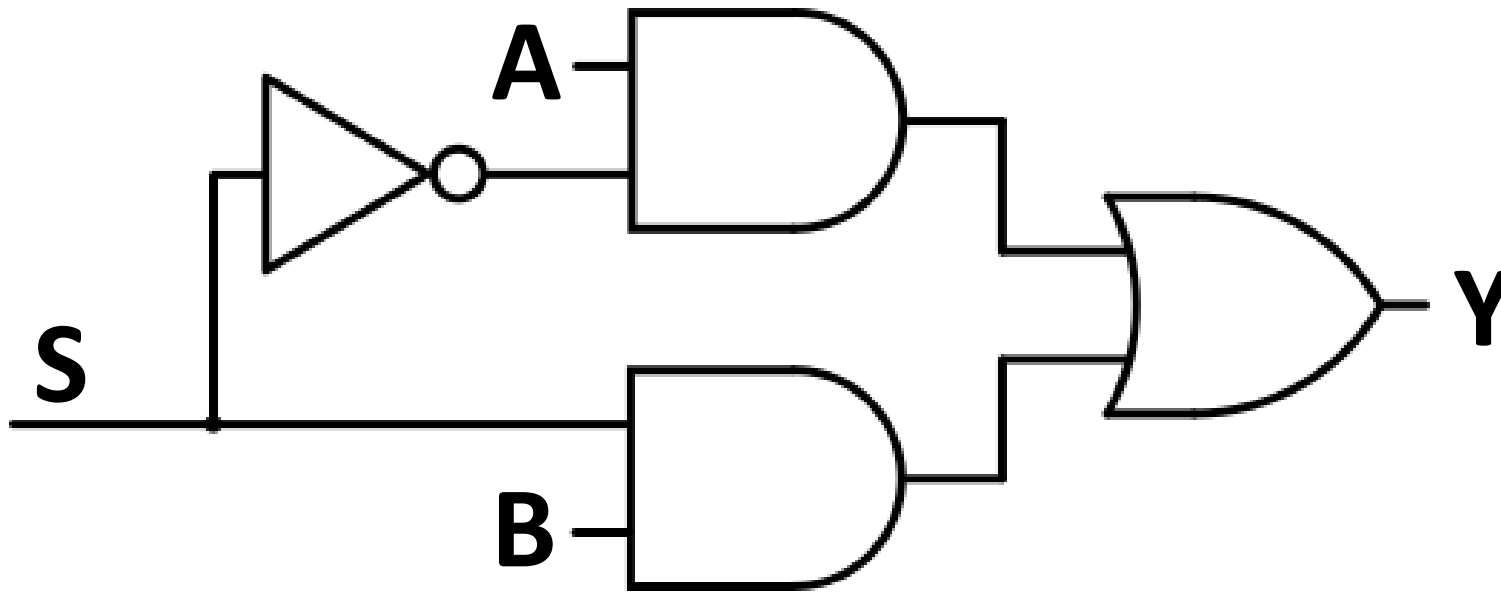
$$Y = S'A + SB$$

S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	



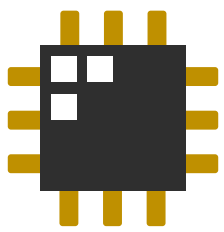
More complex logic

- We can make combinations of logic gates to make more complex logic!



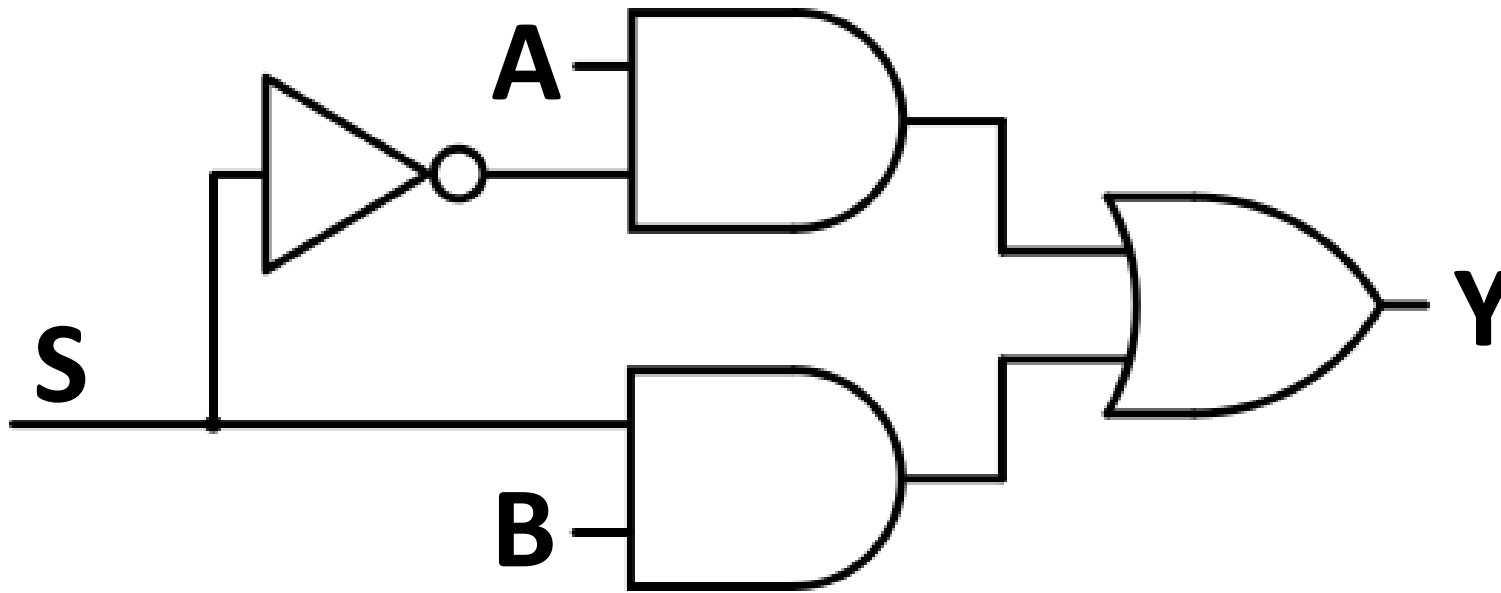
$$Y = S'A + SB$$
$$(1)A + (0)B$$

S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	



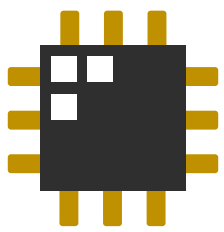
More complex logic

- We can make combinations of logic gates to make more complex logic!



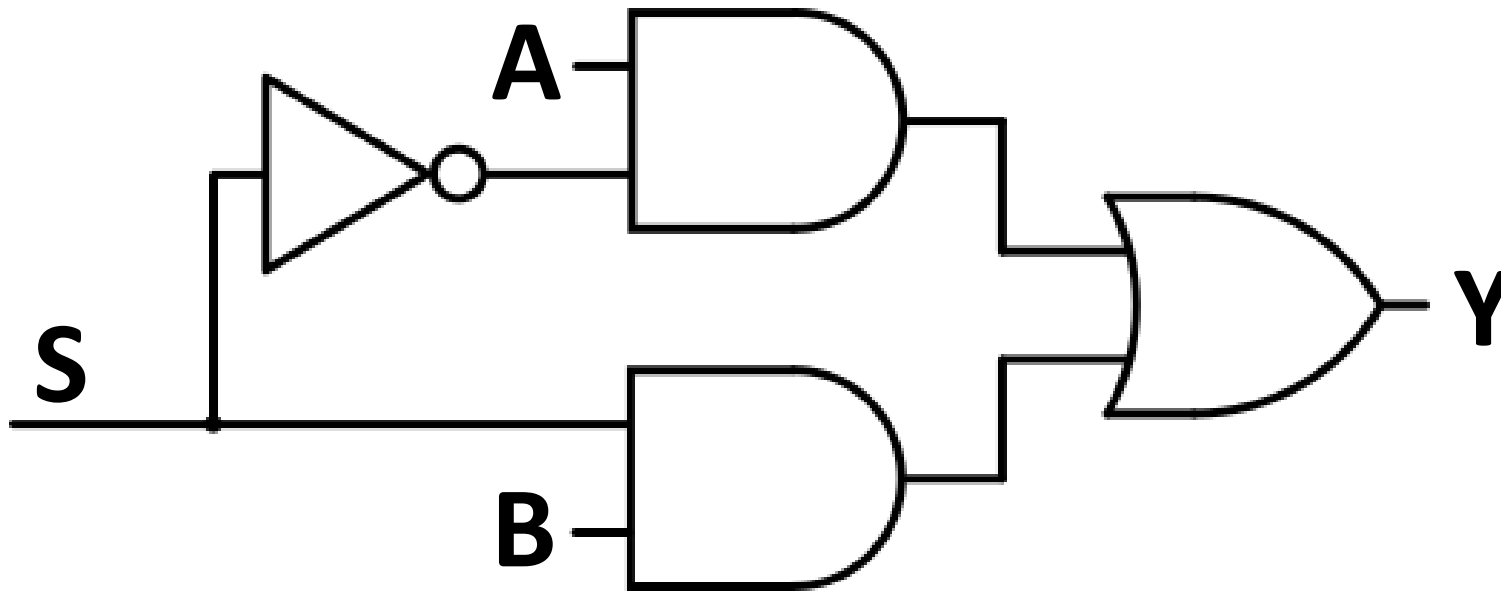
$$Y = S'A + SB$$
$$(0)A + (1)B$$

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	
1	0	1	
1	1	0	
1	1	1	



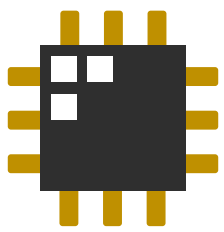
More complex logic

- We can make combinations of logic gates to make more complex logic!



$$Y = S'A + SB$$

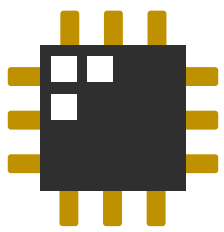
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- Boolean expressions and truth tables represent the same thing
 - You can derive one from the other and vice versa!
- Write the equivalent Boolean expression for the truth table on the left.
 - **Tip:** Write the conditions that needs to be satisfied to make the output 1



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$A'B'C'$

_____?

_____?

_____?

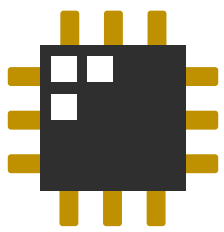
- For the first row, we say **Q** becomes 1 if A is 0 **AND** B is 0 **AND** C is 0

$A' \text{ AND } B' \text{ AND } C'$

- If something *needs to be 0* to satisfy a condition, that implies the **NOT** logic

- If a *specific combination of inputs* need to be satisfied, that implies the **AND** logic

- Looking at a single row

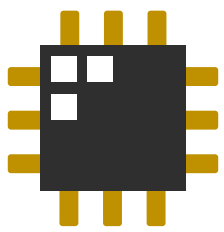


Truth tables to Boolean expression

A	B	C	Q	
0	0	0	1	$A'B'C'$
0	0	1	1	$A'B'C$
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	$AB'C$
1	1	0	0	
1	1	1	1	ABC

- If *either of the several combinations of inputs* can be satisfied to make the output 1, that implies the **OR** logic
 - Looking at multiple rows

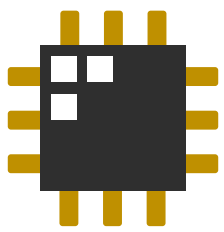
$$Q = A'B'C' + A'B'C + AB'C + ABC$$



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Practice: Write the equivalent Boolean expression of the truth table on the left



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Practice: Write the equivalent Boolean expression of the truth table on the left

$A'B'C'$

$A'B'C$

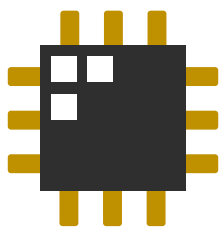
$A'BC'$

$A'BC$

ABC'

ABC

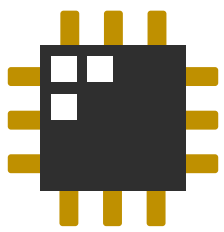
$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Boolean expression minimization

- Once we start writing out expressions, we start to find some ‘patterns’
 - The fact that the expression resembles a math equation makes it tempting to ‘group up terms’ and attempt to ‘simplify’...

$$Q = A'B'C' + A'B'C + AB'C + ABC$$



Boolean expression minimization

- Once we start writing out expressions, we start to find some ‘patterns’
 - The fact that the expression resembles a math equation makes it tempting to ‘group up terms’ and attempt to ‘simplify’...

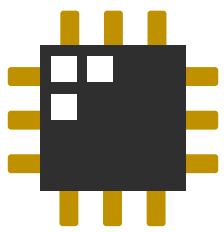
$$Q = A'B'C' + A'B'C + AB'C + ABC$$

$$Q = A'B'(C' + C) + AC(B' + B)$$

Can we do this?

Yes, yes we can. 😊

Let's formalize some theorems first



Boolean theorems

- For single variables: Let's say **A** is a variable

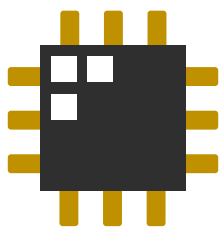
- $A(1) = A$ (Any variable ANDed to 1 is itself)
- $A + 0 = A$ (Any variable ORed to 0 is itself)
- $A(0) = 0$ (Any variable ANDed to 0 is 0)
- $A + 1 = 1$ (Any variable ORed to 1 is 1)
- $AA = A$ (Any variable ANDed to itself is itself)
- $A + A = A$ (Any variable ORed to itself is itself)
- $(A')' = A$ (Complementing twice is itself)
- $A(A') = 0$ (ANDing to its complement is 0)
- $A + A' = 1$ (ORing to its complement is 1)

AND logic

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1



Boolean theorems

- For multiple variables: Let's say **A**, **B**, and **C** are variables

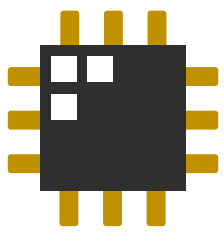
- $AB = BA, A+B = B+A$ (Commutativity)
- $(AB)C = A(BC), (A+B)+C = A+(B+C)$ (Associativity)
- $A(B+C) = AB + AC$ (Distributivity)

Feels like math so far...

- $A+(BC) = (A+B)(A+C)$ (Distributivity, but weirder!)

We can also distribute the complement! (DeMorgan's theorem)

- $(AB)' = A' + B'$; another version is $(A+B)' = A'B'$
(Take note of the change in operator!)

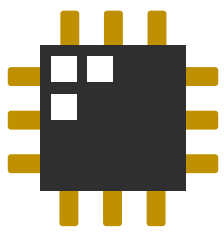


Boolean minimization

- Practice for a bit, write a simplified expression for these terms

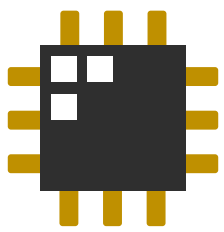
- $A(A+B) = ?$

- $AB + AB' = ?$



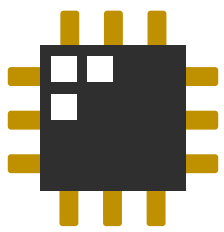
Boolean minimization

- Practice for a bit, write a simplified expression for these terms
 - $A(A+B) = ?$
 - $AA + AB$
 - $A + AB$
 - $A(1+B)$
 - A
 - $AB + AB' = ?$



Boolean minimization

- Practice for a bit, write a simplified expression for these terms
 - $A(A+B) = ?$
 - $AA + AB$
 - $A + AB$
 - $A(1+B)$
 - A
 - $AB + AB' = ?$
 - $A(B+B')$
 - $A(1)$
 - A



Boolean expression minimization (Part 1)

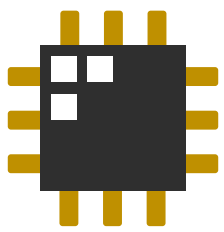
- Let's go back to this!

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

$$Q = A'B'(C' + C) + AC(B' + B)$$

Can we do this?

Yes, yes we can. 😊



Boolean expression minimization (Part 1)

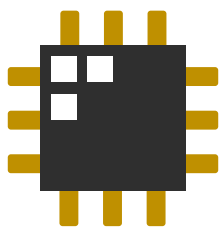
$$Q = A'B'C' + A'B'C + AB'C + ABC$$

$$Q = A'B'(C' + C) + AC(B' + B)$$

$$Q = A'B'(1) + AC(1)$$

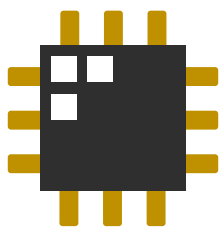
$$Q = A'B' + AC$$

Final answer



Boolean expression minimization (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

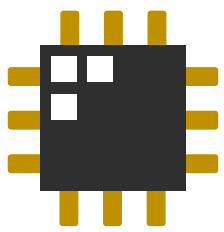


Boolean expression minimization (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$$

$$Q = A'B' + A'B + AB$$



Boolean expression minimization (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$$

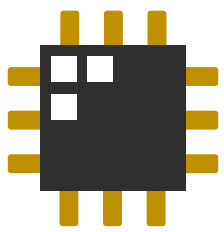
$$Q = A'B' + A'B + AB$$

$$Q = A'B' + A'B + A'B + AB$$

$$Q = A'B' + A'B + A'B + AB$$

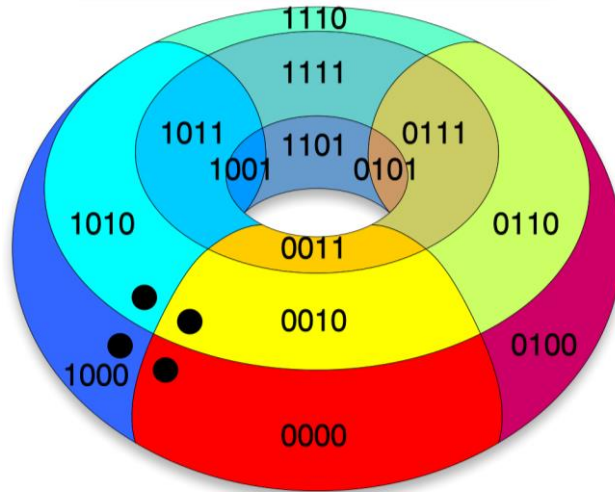
$$Q = A'(B' + B) + B(A' + A)$$

$$Q = A' + B \quad \textit{Final answer}$$



Boolean minimization

- How do we know if it's already in the most simplified expression?
 - There are techniques taught in EEE 143 that will allow you to confirm whether it cannot be simplified further... We won't cover it here.

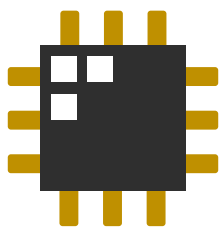


• 0000	0100	1100	• 1000
0001	0101	1101	1001
0011	0111	1111	1011
• 0010	0110	1110	• 1010

It looks like this...

Though probably not drawn as a doughnut ☹️

- When we give minimization problems like this, we'll make sure the 'patterns' are easy to find. 😊
- We'll also try not to give a very long expression to start with.

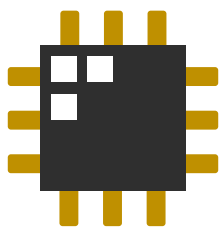


Logical equivalence

- Are we sure that the simplified expression is equivalent to the original expression?
 - As long as you followed the theorems correctly, it should 😊



- You can write the truth table again using the simplified expression. The resulting truth table should be the same!



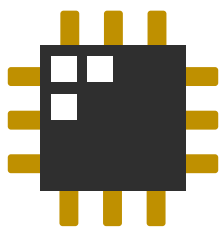
Logical equivalence (Part 1)

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$Q = A'B' + AC$$

A	B	C	Q
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	



Logical equivalence (Part 1)

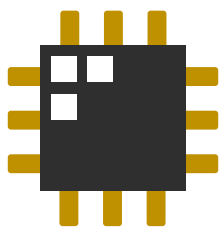
$$Q = A'B'C' + A'B'C + AB'C + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Tables are the same
Logically equivalent!

$$Q = A'B' + AC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



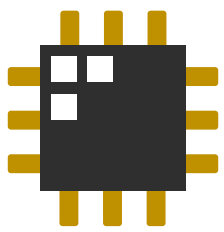
Logical equivalence (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = A' + B$$

A	B	C	Q
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	



Logical equivalence (Part 2)

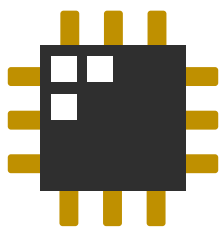
$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A' + B$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

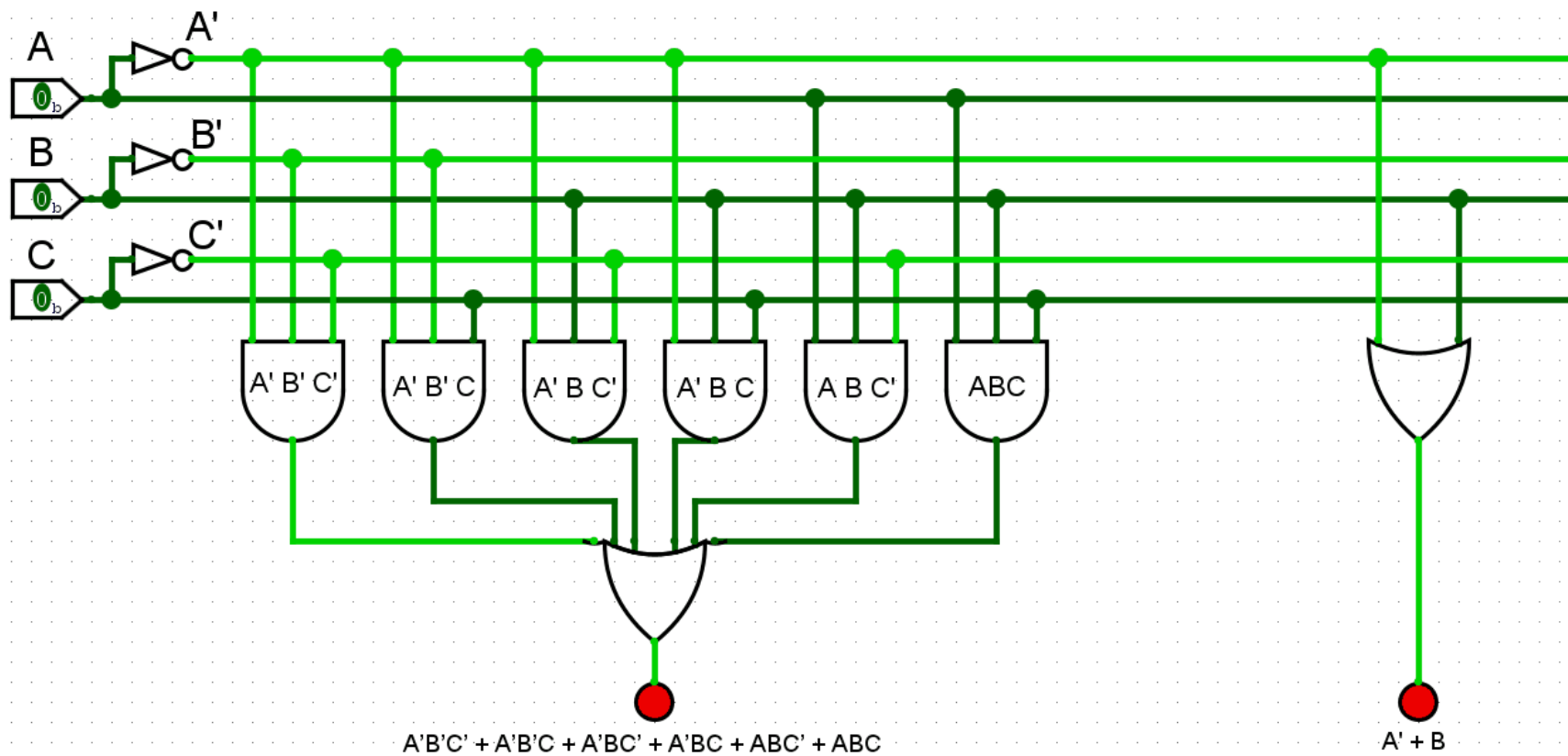
Tables are the same
Logically equivalent!

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Logical equivalence

- If the truth tables are the same, they are functionally equivalent!
- Simulation demo:



**Fewer logic gates.
More efficient!**

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A' + B$$